

Engenharia de Software com Vetores: Busca e Encapsulamento em C

Guia Explicativo – Semana 3: Vetores:
Aplicações e Boas Práticas (Parte 2)

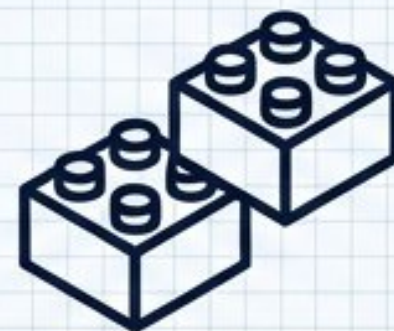
Disciplina: TEC0790 – Estruturas de Dados | Curso Técnico em Desenvolvimento de Sistemas

Objetivos de Aprendizagem da Semana



1. Algoritmos de Busca

Resolver problemas de busca simples implementando a Busca Sequencial.



2. Modularização (C)

Encapsular operações complexas em funções específicas e reutilizáveis.



3. Padrões Profissionais

Melhorar a organização, evitar duplicação e adotar boas práticas de legibilidade.

O Universo de Operações com Vetores

Busca



Encontrar a posição exata de um elemento específico (chave) dentro da estrutura.

Contagem



Rastrear quantas vezes um valor alvo ou condição aparece na coleção.

Filtragem



Isolar elementos que atendam a uma regra (ex: apenas números pares).

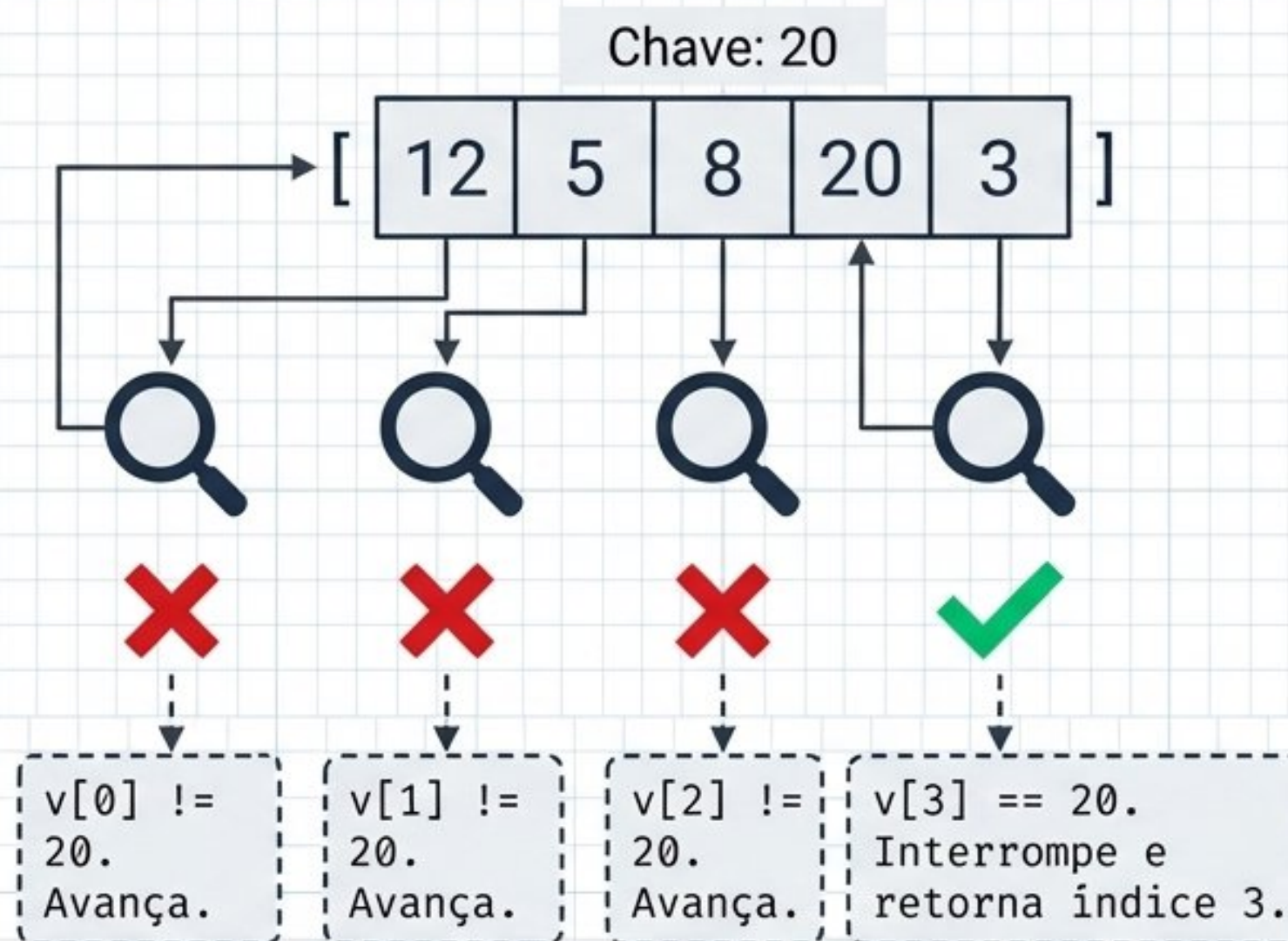
Agregação



Consolidar dados em um único resultado (ex: Mínimo, Máximo, Média).

Foco da Semana 3: Dominar as lógicas de Busca e Contagem.

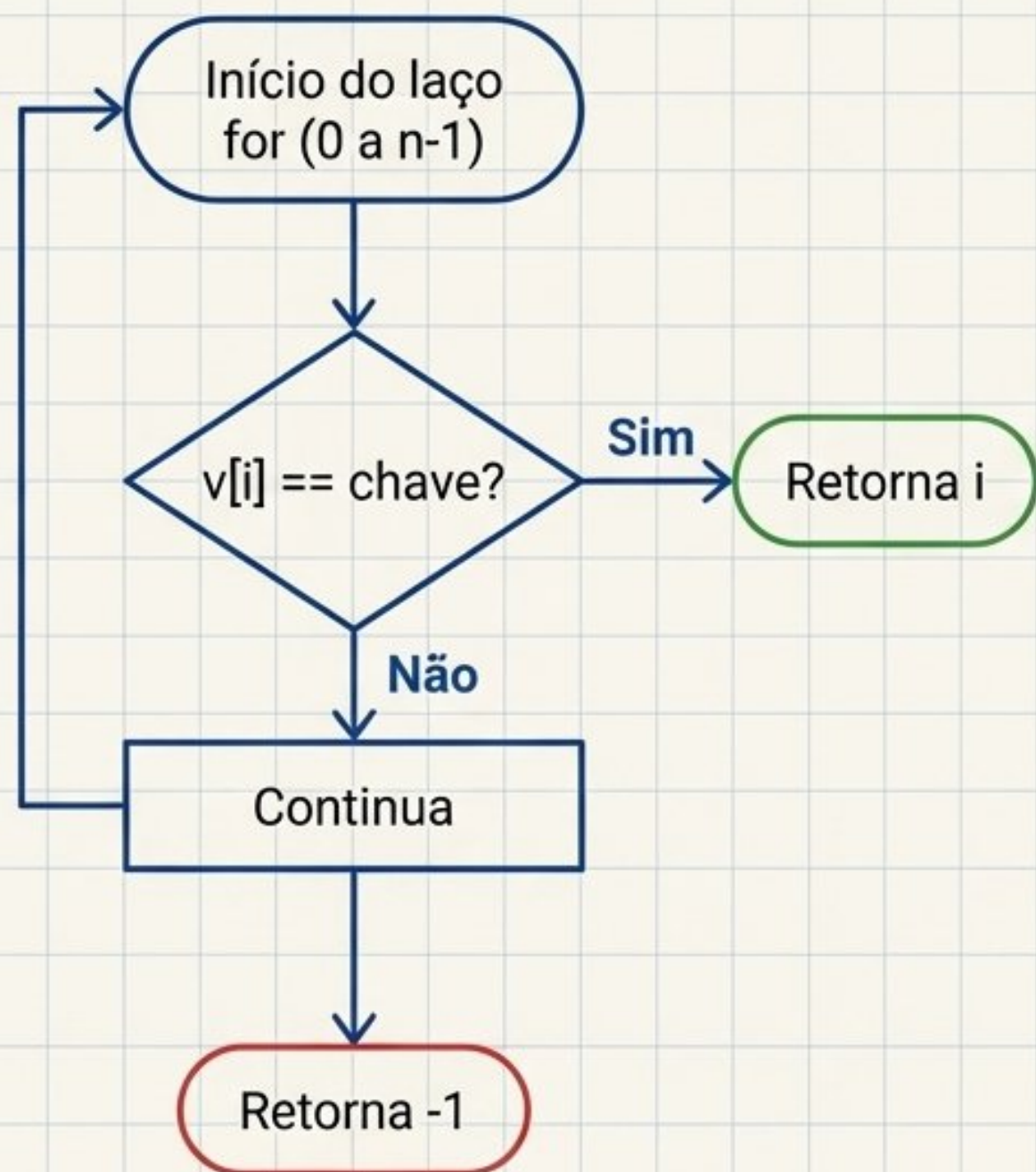
A Anatomia da Busca Sequencial



E se a chave fosse 99?

O laço percorreria até o limite $(n-1)$. Ao não encontrar nada, o algoritmo adota um padrão universal: Retornar `-1` (já que não existe índice negativo em C).

Da Lógica ao Código: Busca Sequencial



Quebra a execução da função imediatamente. Não desperdiça processamento.

```

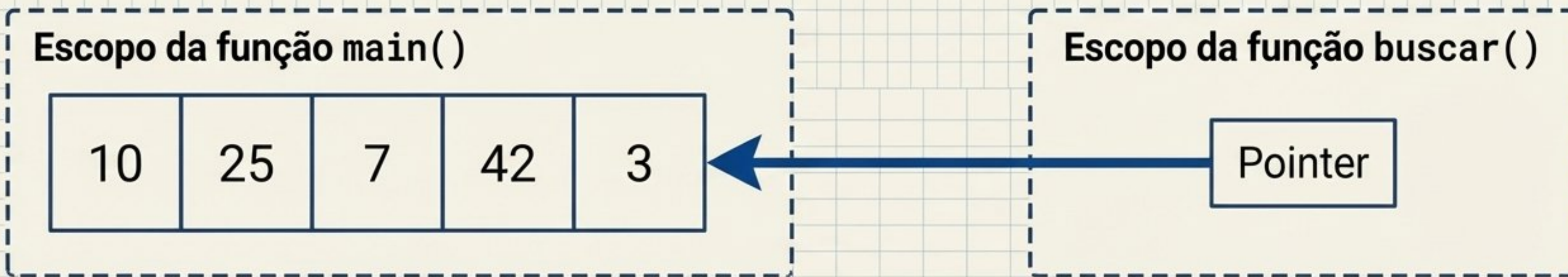
int buscar(int v[], int n, int chave) {
    for (int i = 0; i < n; i++) {
        if (v[i] == chave) {
            return i; // Encontrou: parada imediata
        }
    }
    return -1; // Não encontrou: valor especial
}
  
```

Só é atingido se o laço terminar sem sucesso.

O Paradigma da Modularização

	Código Monolítico (Amador)	Código Modularizado (Profissional)
Estrutura	Tudo escrito dentro da função <code>main()</code> .	Dividido em funções específicas (<code>ler_vetor</code> , <code>imprimir_vetor</code> , <code>buscar_valor</code>).
Repetição (DRY)	Copia e cola o mesmo laço <code>for</code> várias vezes.	O laço é escrito uma vez na função e chamado múltiplas vezes.
Testabilidade	Um erro quebra o programa inteiro. Difícil de isolar.	Funções podem ser testadas isoladamente antes de integrar.
Legibilidade	Exige leitura linha por linha para entender o que o bloco faz.	Código autodocumentado pelos nomes das funções. O <code>main()</code> vira apenas um gerente.

O Segredo do C: Passagem por Referência Implícita

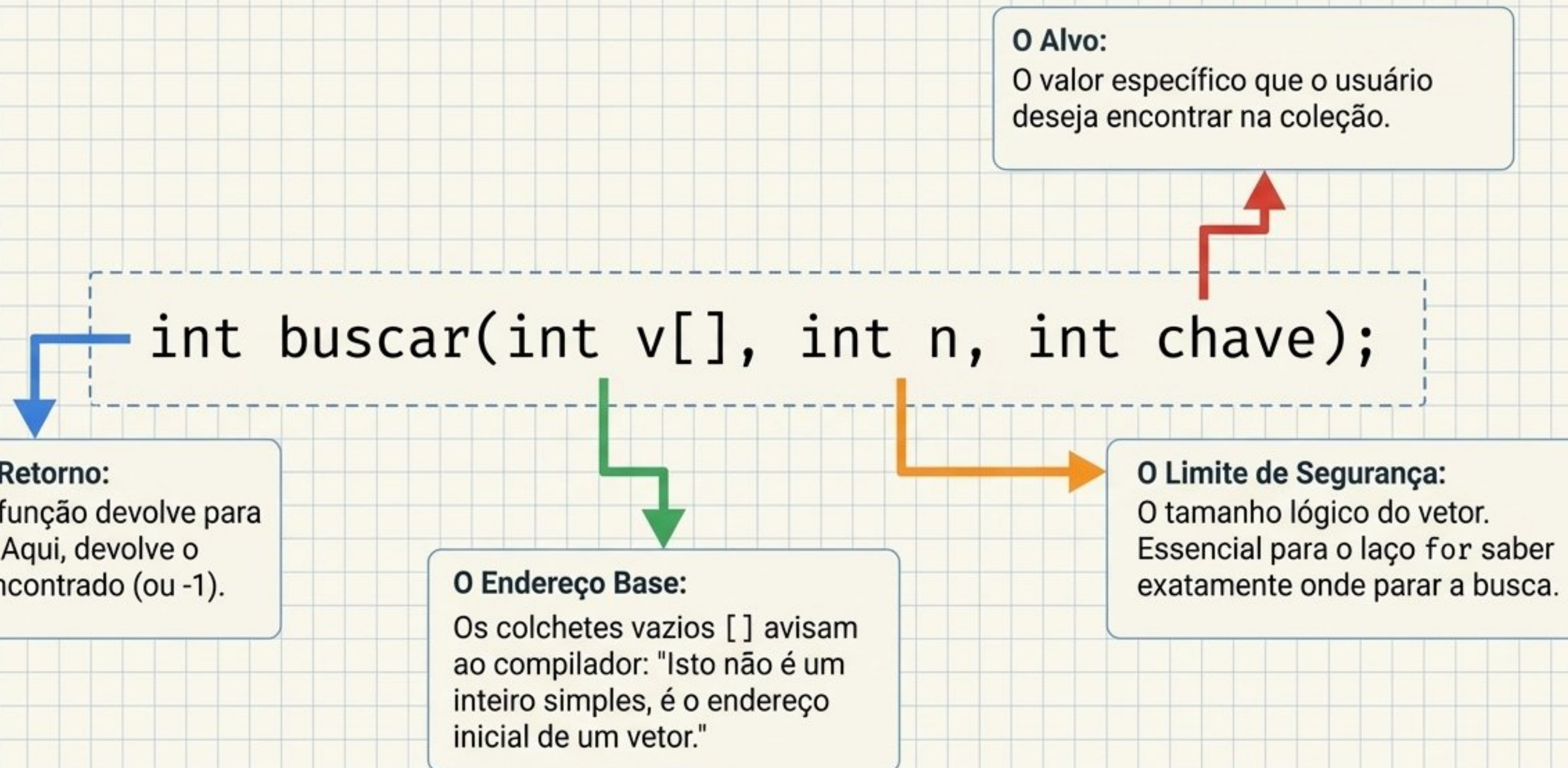


- **A Ilusão da Cópia:** Em C, vetores nunca são copiados para dentro das funções.
- **Acesso Direto:** A função recebe apenas o endereço de memória inicial (um ponteiro) e acessa os dados originais.



Atenção: Como a função só recebe o endereço de início, ela é cega para o final do vetor. É por isso que o tamanho 'n' deve ser obrigatoriamente passado como parâmetro.

Dissecando o Protótipo da Função



Padrão de Projeto: Contagem de Ocorrências

[7 2 7 9 7]

Alvo: 7

03

Diferente da busca (que para no primeiro acerto), a contagem exige que o vetor inteiro seja percorrido.

```
int contar_ocorrencias(int v[], int n, int alvo) {  
    int contador = 0; // Variável acumuladora  
  
    for (int i = 0; i < n; i++) {  
        if (v[i] == alvo) {  
            contador++; // Incrementa a cada acerto  
        }  
    }  
  
    return contador; // Só retorna após o laço terminar  
}
```


Boas Práticas de Engenharia de Código

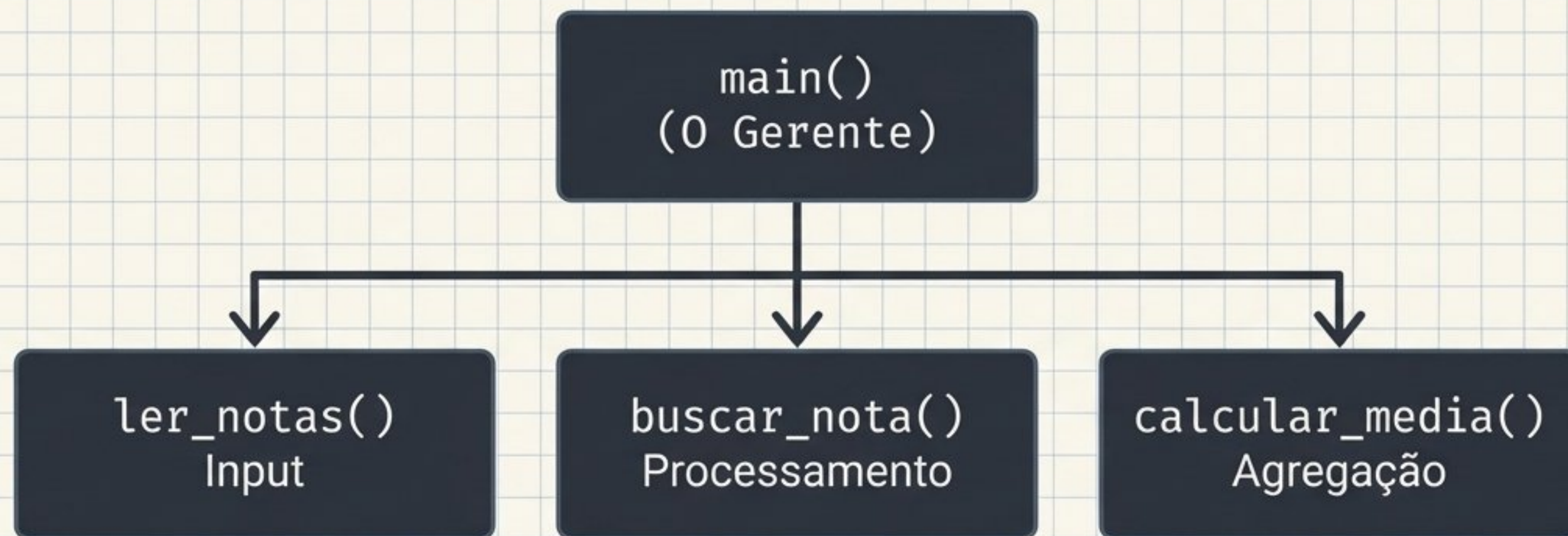
O Que Fazer

- ✓ **Responsabilidade Única:** Uma função deve fazer apenas uma coisa. Se ela busca, não deve imprimir o resultado na tela. Apenas retorne o valor.
- ✓ **Nomes Descritivos:** Use nomes que expliquem a ação, como ``contar_pares`` ou ``buscar_nota``, como se o código fosse autodidático.
- ✓ **Comentários Estratégicos:** Comente o 'porquê' de lógicas complexas, não o 'o que' (o código já diz o que).

O Que NÃO Fazer

- ✗ **Nomes Genéricos:** Evite ``func1``, ``x``, ``variavel``.
- ✗ **Duplicação (Violar o DRY):** Se você precisou copiar e colar um bloco de código, ele deveria ser transformado em uma função.
- ✗ **Retornos Múltiplos ou Confusos:** Tentar fazer a função de busca retornar várias posições de uma vez usando métodos não padronizados.

Estudo de Caso: Mini-Sistema de Notas



Em um sistema real de cadastro de notas ou idades, o main apenas exibe o menu para o usuário e delega o trabalho pesado para as funções especializadas. O vetor "viaja" por referência entre elas.

Laboratório Prático da Semana

Missão de Implementação

[] Implementar a Lupa: Criar a função `int buscar(int v[], int n, int chave)` garantindo o retorno de -1 em falhas.

[] Implementar o Contador: Criar a função para contar a ocorrência de valores específicos.

[] Orquestração: Construir um programa principal (main) que lê a quantidade de alunos N, lê o vetor de notas/idades, e permite ao usuário testar ambas as funções.

Teste seus limites: O que acontece se você buscar uma nota que não foi cadastrada?

Atividade Verificadora (Desafio Final)



Buscas Repetidas

O Problema

Sistemas reais não encerram após uma única busca. Eles aguardam novas ordens.

A Missão

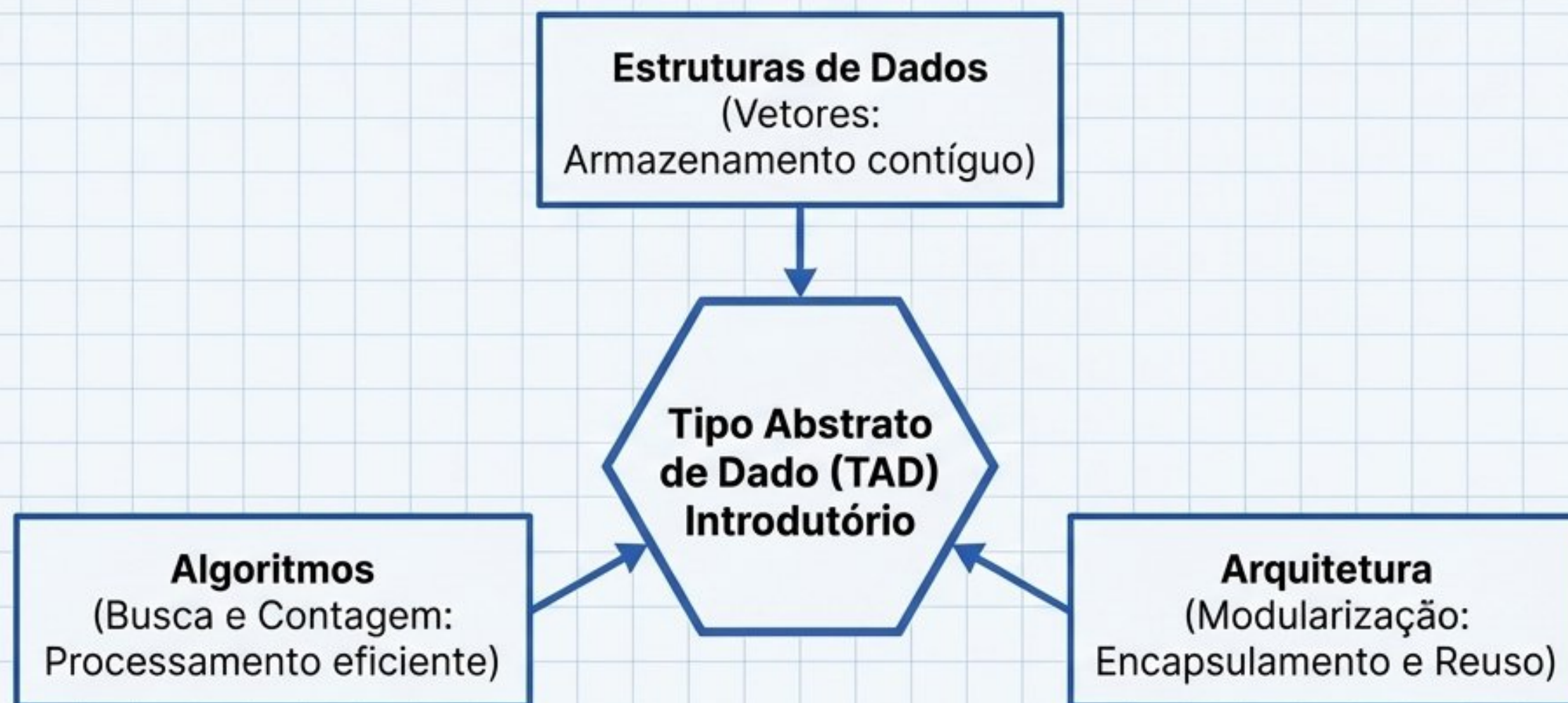
Entregar um programa modularizado que leia um vetor de inteiros e, através de um laço `while` no `main`, permita ao usuário buscar valores indefinidamente até que ele digite um comando de saída (ex: buscar por valor negativo).

✓ Uso correto da busca sequencial.

✓ Modularização rigorosa (busca separada do `main`).

✓ Retorno posicional e tratamento do resultado "Não encontrado".

Síntese: A Construção de Software Robusto



Dominar vetores não é apenas saber como declará-los, mas como manipulá-los de forma eficiente e encapsulada. Ao isolar operações complexas em funções, você acabou de plantar a semente para o conceito de TAD, preparando o terreno para as Estruturas Heterogêneas (Structs) e Listas das próximas semanas.